

L'ordinateur

- Le terme 'ordinateur' est d'origine biblique
 - « celui qui met en ordre » (divin)
 - ordonnateur
 - « entrer dans les ordres », « ordination »
- Il a été proposé par le professeur de *philologie* Jacques Perret dans une lettre datée du 16 avril 1955 en réponse à une demande d'IBM France

Les premiers calculateurs

- Blaise Pascal (1623-1662)
 - 1642 Première calculatrice mécanique



La Pascaline

- Addition, soustraction en base 10
- Leibnitz (1646-1716)
 - 1670 Ajout de l'opération de multiplication, et division

Charles Babbage (1792-1871)

- « *Machine à différence* » :



- un seul algorithme (méthode des différences finies)
- pour les tables numériques pour la navigation en mer
- Résultats gravés sur un plateau de cuivre
 - Machine analytique (1840)
- Lecture sur des cartes perforées
- Différents algorithmes exécutables
- 4 parties
 - Le magasin (la mémoire) = 1000 mots de 50 chiffres = 1,25Mo

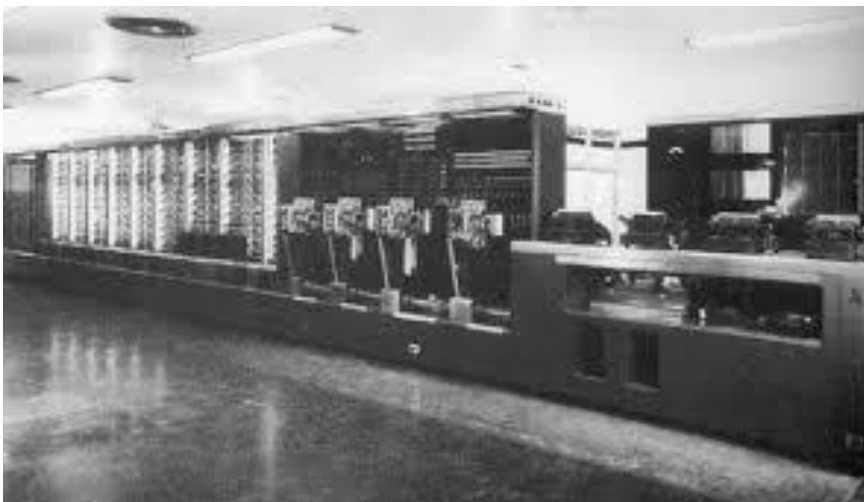
- Le moulin (l'unité de calcul)
- L'entrée (lecteur de cartes perforées)
- La sortie (perforation ou impression)
- 1843 Premier programme informatique
- Ada Lovelace (langage d'assemblage)

Innovations

- Technologiques
 - 1904 John Flemming invente le tube à vide :
 - Interrupteur électronique (pannes fréquentes, encombrant)
 - Réalisations : Alan Turing
- 1943 Premier calculateur numérique
- Resté sous silence pendant 30ans

Machine de Harvard

- « the beginning of the era of the modern computer »
- Ordinateur à **relais**
- 1944 Howard Aiken
 - Mark 1 à **Harvard**
- Entièrement automatique
- Electro-mécanique
- 16m x 2,5m – 4500 Kg
- Démarré par un moteur 5 chevaux
- Utilise un arbre de synchronisation
- 72 mots de 23 chiffres
- Temps de cycle : 6 secondes
- Soit 0.16 Hz



ENIAC - **Premier ordinateur électronique**

- 1946 : Mauchly et Eckert
- Projet de l'US Army

- Capable d'être reprogrammée par branchements
- Vendue à 500 000\$ à 1 unité
- 1 cycle = 200ms soit 5 cycles/sec
- Une multiplication de 2 opérandes à 10 chiffres prenait 14 cycles, soit 2800 ms
- Il est divisé en 30 unités autonomes (dont 20 accumulateurs / additionneurs 10 digits, 1 multiplicateur et 1 "Master Programmer" capable de gérer les boucles).
- ENIAC (1946)
 - 30m de long
 - 2.5m de haut
 - 18 000 tubes à vide
 - 1500 relais
 - 30 Tonnes
 - 140 KW
 - 20 registres de 10 chiffres décimaux
 - 6000 commutateurs pour la programmation



La machine de Von Neumann

- Von Neumann (1903,1957)
 - Il se greffe au projet ENIAC après 1946
- Idées
 - On peut représenter le programme sous une forme numérique et le ranger en mémoire comme les données (plus de programmation des commutateurs)
 - Introduction de l'arithmétique **binaire**!
 - Ancêtre à la base des architectures actuelles

- Résultat : l'IAS (1952)
 - Premier ordinateur à programme enregistré
- Mémoire séparée du CPU

5 parties

- La mémoire
- L'UAL
- L'unité de commande
- Les entrées
- Les sorties

L'assembleur (premier langage machine)

- 1947 : Amélioration du traitement binaire :
le langage *Assembleur* est inventé par Nathaniel Rochester

Invention du transistor

1948 : Invention du **transistor** aux Bell Labs (Bardeen, Shockley)

- Interrupteur commandé électroniquement
- Beaucoup moins encombrant que le tube
- Concepts architecturaux
 - Programmation en langage machine
 - Microprogrammation
 - Représentation des nombres en virgule flottante
- Logicielles
 - Apparition des systèmes d'exploitation
 - Programmation en assembleur
 - Premier langage de haut niveau : le FORTRAN (premier langage de haut niveau)

Le début des grandes firmes

- IBM
 - (7090, 7094, 7030...)
- Bull
 - (PDP-1, PDP-8...)
- DEC
 - (TX-0, TX-2...)
- CDC
 - (6600, 6700, Cray-1...)

Premiers circuits intégrés (sur silicium R. Noyce)

- Innovations technologiques
- Moindre encombrement
- Moindre consommation
- Gain en performances

Naissance d'Intel, le premier microprocesseur

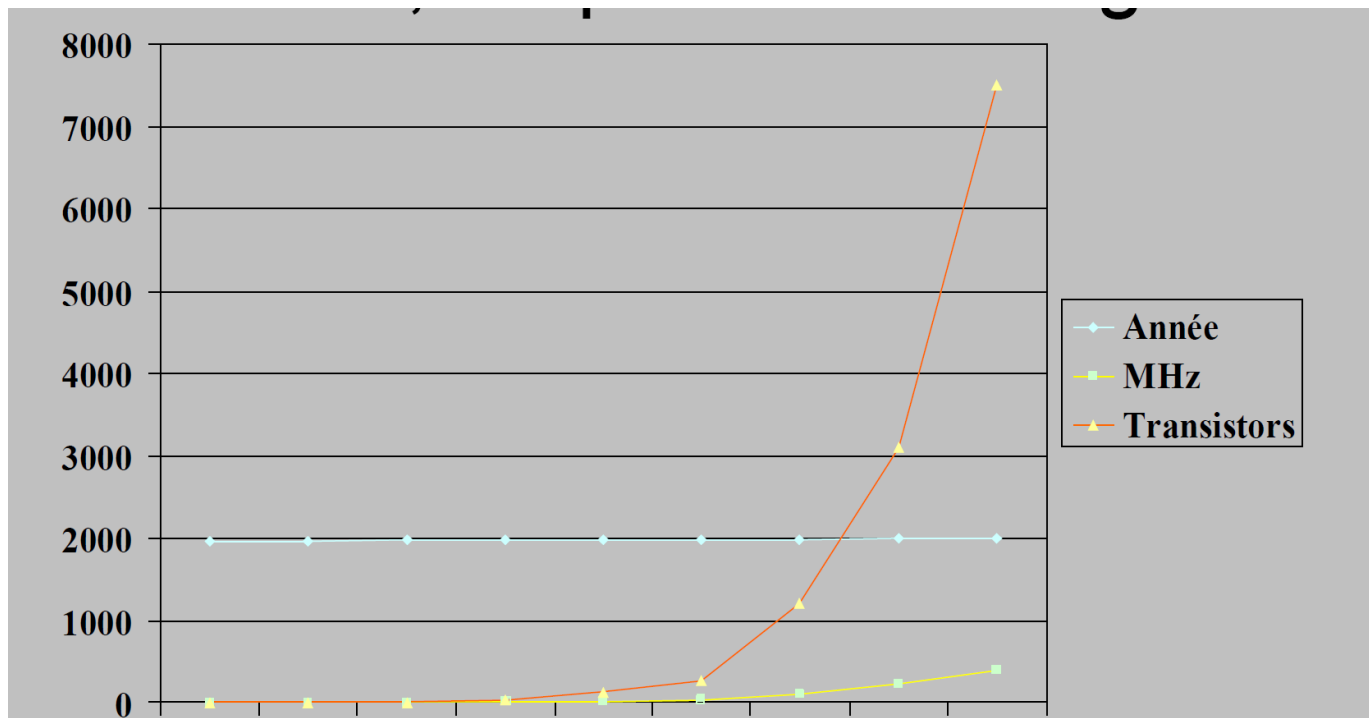
- 1968 R. Noyce, G. Moore et A. Rock fondent Intel Corporation
 - 1970 première UC composée de 2300 transistors (LSI) : le 4004 processeur 4bits
 - 1973 le 8008 processeur 8bits
 - 1978 le 8086 processeurs 16 bits
 - 1971 : Premier microprocesseur Intel 4004
- Microprocesseur 4 bits 108 KHz 60000 Instructions par seconde
2300 transistors

4ème génération

Les VLSI

(1980, maintenant)

- Densité d'intégration augmente : VLSI
 - Dizaine de milliers -> millions de transistors sur une puce
- Passage du centre de calcul à l'ordinateur personnel
- Innovations architecturales
 - réseaux d'ordinateurs
 - Concepts de parallélisme développés
- 1977 Apple puis **Apple II** (Steve Jobs)
 - Plusieurs millions d'exemplaires
- 1981 l'**IBM PC** (personal computer)
 - Intel 8088
 - MS-DOS (microsoft)
 - Plans diffusés gratuitement => clonage
- 1980 Architecture RISC dominante
- 1990 UC Superscalaires



Unités de taille

En informatique, la grandeur de base est le **bit** ([binary](#) digit). Un bit est un élément pouvant être égal à 0 ou à 1 (deux valeurs possibles donc).

Un fichier est un ensemble de bits. Un ensemble de bits forme ce qu'on appelle un mot [binaire](#).

Le langage [binaire](#) est le seul que l'ordinateur comprend.

Pour simplifier les grandeurs, on utilise des multiples de l'octet (un ensemble de 8 bit), à ne pas confondre avec la définition anglaise de l'octet qui s'écrit **Byte** (avec le B majuscule).

Un bit ne suffisant pas pour exprimer [toutes les tailles](#) de fichiers disponibles, des unités de mesures (comme le centimètre, le mètre et le kilomètre par exemple) ont été mises en place :

- Le kilo-octet (Ko) : 1 Ko équivaut à 1000 octets.
- Le méga-octet (Mo) : 1 Mo = 1000 Ko.
- Le giga-octet (Go) : 1 Go = 1000 Mo.
- Le téra-octet (To) : 1 To = 1000 Go.

Pour garder nos anciennes normes qui voulaient que les unités de mesures soient des puissances de 2 (1 Ko = 1024 octets par exemple), d'autres unités de mesures (peu utilisées) ont été inventées :

- Le kibi-octet : (Kio) 1 Kibi-octet équivaut à 1024 octets.
- Le mébi-octet (Mio) : 1 mébi-octet équivaut à 1024 Kio.
- Le gibi-octet (Gio) : 1 gibi-octet équivaut à 1024 Mio.
- Le tébi-octet (Tio) : 1 tébi-octet équivaut à = 1024 Gio.

Unités d'affichage

Lorsque l'on parle d'affichage, on pense souvent de pixels. Un pixel est le plus petit carré affichable sur votre écran. Comme il n'est pas de [dimensions](#) identiques, on rajoute souvent l'unité dpi (dot per inch ou ppp pour points par pouce ou pixels par pouce). Plus le nombre de pixels par pouce est élevé, [meilleure est](#) la qualité.

En consultant des sites de jeux vidéo, vous aurez pu croiser la notion de fps (frame per second) ou encore ips (image par seconde), qui signifient toutes les deux le nombre d'images par seconde qui s'affichent.

Unités d'impression

En comparant des [imprimantes](#), vous pouvez tomber sur le nombre de pages par minute (ppm) que l'imprimante est capable d'imprimer.

Unités de débit

Pour mesurer les débits (notamment dans le domaine des réseaux et connexions [internet](#)), il est très courant d'utiliser le bit/s (bits par seconde ou bps).

Codage Binaire, Octet

Le langage binaire vient de la notation de contacteurs, notée 0 pour **faux** (quand le courant ne passe pas) et 1 pour **vrai** (quand le courant passe).

Ce codage est nommé **base binaire** et est utilisé par l'ordinateur pour comprendre et traiter les données. Les humains travaillent avec dix chiffres, c'est la notation décimale (base 10).

Bit

Le bit vient du terme Binary Digit (soit 0 soit 1 en langage binaire). On travaille donc avec les puissances de 2. C'est ainsi qu'avec 3 bit, on peut avoir 2^3 (8) dans notre cas. Il y a donc 8 possibilités.

Valeur décimale	Bit 3 (2^2)	Bit 2 (2^1)	Bit 1 (2^0)	Calcul
0	0	0	0	$0 = 0 * (2^2) + 0 * (2^1) + 0 * (2^0)$
1	0	0	1	$1 = 0 * (2^2) + 0 * (2^1) + 1 * (2^0)$
2	0	1	0	$2 = 0 * (2^2) + 1 * (2^1) + 0 * (2^0)$
3	0	1	1	$3 = 0 * (2^2) + 1 * (2^1) + 1 * (2^0)$
4	1	0	0	$4 = 1 * (2^2) + 0 * (2^1) + 0 * (2^0)$
5	1	0	1	$5 = 1 * (2^2) + 0 * (2^1) + 1 * (2^0)$
6	1	1	0	$6 = 1 * (2^2) + 1 * (2^1) + 0 * (2^0)$
7	1	1	1	$7 = 1 * (2^2) + 1 * (2^1) + 1 * (2^0)$

Le code binaire est un code pondéré car il s'écrit sous forme d'additions de puissances de 2. Pour passer du binaire à la valeur décimale correspondante (en base 10), on additionne les nombres en les élevant à leurs puissances de 2 respectives.

Binaire réfléchi (code GRAY)

Il existe une autre forme (non pondérée cette fois) du code binaire, le code GRAY (aussi appelé binaire réfléchi). On ne peut modifier qu'un seul bit à la fois lorsque l'on passe d'un nombre n à un nombre $n+1$ et il existe une symétrie verticale :

Valeur	Bits		
0	0	0	0
1	0	0	0
2	0	0	1
3	0	0	1
4	0	1	1
5	0	1	1
6	0	1	0
7	0	1	0
Axe de symétrie			
8	1	1	0
9	1	1	0
10	1	1	1
11	1	1	1
12	1	0	1
13	1	0	1
14	1	0	0
15	1	0	0

Octet

Un octet est un ensemble de 8 bit. Il permet de stocker des nombres pouvant aller jusqu'à 255 car il offre 2^8 soit 256 possibilités. C'est une unité très couramment utilisée en informatique.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Quartet

Peu utilisé en informatique, le quartet est un ensemble de 4 bit.

Octet, Kilo-octet, Méga-octet

- Un octet est un ensemble de 8 bit.
- Un Kilo-octet (Ko) vaut 1000 octets.
- Un Méga-octet (Mo) vaut 1000 Kilo-octets
- Un Giga-octet (Go) vaut 1000 Méga-octets
- Un Téra-octet (To) vaut 1000 Giga-octets
- Le Péta-octet (Po) vaut 1000 Téra-octets

Depuis 1998, la commission électrotechnique internationale a normalisé les unités suivantes en puissances de 2 :

- Le kibi-octet (Kio) qui est égal à 1024 octets.
- Le Mébi-octet (Mio) qui est égal à 1024 Kibi-octets.
- Le Gibi-octet (Gio) qui est égal à 1024 Mébi-octets.
- Le Tébi-octet (Tio) qui est égal à 1024 Gibi-octets.
- Le Pébi-octet (Pio) vaut 1024 Tébi-octets.

Le langage hexadécimal

Le langage hexadécimal est un langage en base 16 venu freiner l'accroissement en longueur des nombres binaires. Pour composer ce langage, les 10 premiers chiffres suivis des six premières lettres de l'alphabet ont été utilisés :

0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

No tation décimale															
	0	1	2	3	4	5									
No tation binaire	0	1	00	01	10	11	000	001	010	011	100	101	110	111	
No tation hexadécimale															

[Conversion décimal vers hexadécimal :](#)

Méthode des divisions successives (exemple pour 36 en décimal) :

36	16
4	2

On procède de la même manière qu'en binaire, à savoir que l'on remonte les valeurs de bas en haut. On obtient 36 (décimal) = 24 (hexadécimal). On peut retrouver et contrôler notre résultat en constatant que : $2 * 16^1 + 4 * 16^0 = 36$

De l'hexadécimal au binaire :

Conservons notre nombre de 24 en hexadécimal (36 en décimal) :

On utilise ce qu'on appelle le code 1248 (se lit "un deux quatre huit").

2	4
0010	0100

2 = 0010 en binaire, et 4 est égal à 0100 en binaire. On en déduit ensuite le nombre binaire final qui est la concaténation de nos deux nombres binaires :

0010 0100

Ce nombre est égal à 36 en décimal, nous avons donc converti notre nombre hexadécimal en binaire, puis en décimal.

Convertir des nombres binaires et décimaux

Pour convertir un nombre décimal en binaire, on a deux possibilités :

- La méthode des divisions successives (que nous allons voir en premier).
- La méthode des puissances.

On divise par deux le nombre jusqu'à ce que le nombre trouvé soit 1. Voici un exemple avec le nombre 27 :

27	2				
1 (5)	13	2			
	1 (4)	6	2		
		0 (3)	3	2	
			1 (2)	1 (1)	

On part ensuite du sens inverse (on prend les bits numérotés successivement 1; 2; 3; 4; 5 en commençant par le chiffre en rouge, bit de poids faible). Donc $27 = 11011$.

Deuxième méthode : on utilise les puissances :

$$27 = 1 * 2^0 + 1 * 2^1 + 1 * 2^3 + 1 * 2^4$$

Tableau des caractères ASCII simple

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	\$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

Tableau des caractères ASCII étendu

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	Ç	160	A0	á	192	C0	Ł	224	E0	α
129	81	ù	161	A1	í	193	C1	ł	225	E1	β
130	82	é	162	A2	ó	194	C2	ṽ	226	E2	Γ
131	83	â	163	A3	ú	195	C3	ṭ	227	E3	π
132	84	ä	164	A4	ñ	196	C4	—	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	†	229	E5	σ
134	86	å	166	A6	ª	198	C6	‡	230	E6	μ
135	87	ç	167	A7	º	199	C7	‡	231	E7	τ
136	88	ê	168	A8	¿	200	C8	Ł	232	E8	Φ
137	89	ë	169	A9	ƒ	201	C9	ƒ	233	E9	Θ
138	8A	è	170	AA	¬	202	CA	Ł	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	ƒ	235	EB	Θ
140	8C	î	172	AC	¼	204	CC	‡	236	EC	∞
141	8D	ì	173	AD	ı	205	CD	=	237	ED	∞
142	8E	Ā	174	AE	«	206	CE	‡	238	EE	ε
143	8F	Ă	175	AF	»	207	CF	±	239	EF	Π
144	90	É	176	B0	⋯	208	DO	Ł	240	FO	≡
145	91	æ	177	B1	⋮	209	D1	ƒ	241	F1	±
146	92	Æ	178	B2	⋮	210	D2	π	242	F2	≥
147	93	ô	179	B3		211	D3	Ł	243	F3	≤
148	94	ö	180	B4	†	212	D4	Ł	244	F4	[
149	95	ò	181	B5	‡	213	D5	ƒ	245	F5	]
150	96	û	182	B6	‡	214	D6	π	246	F6	÷
151	97	ù	183	B7	π	215	D7	‡	247	F7	≈
152	98	ÿ	184	B8	ƒ	216	D8	‡	248	F8	•
153	99	Ö	185	B9	‡	217	D9	ƒ	249	F9	•
154	9A	Û	186	BA		218	DA	ƒ	250	FA	·
155	9B	¢	187	BB	π	219	DB	■	251	FB	√
156	9C	£	188	BC	Ł	220	DC	■	252	FC	π
157	9D	¥	189	BD	Ł	221	DD	■	253	FD	z
158	9E	ℳ	190	BE	Ł	222	DE	■	254	FE	■
159	9F	ƒ	191	BF	ƒ	223	DF	■	255	FF	□

Opérations en binaire

Addition binaire

L'addition en binaire se fait de la même manière qu'en notation décimale: On commence à additionner les bits de droite (ceux appelés bits de poids faible) puis on pose une ou plusieurs retenue(s) lorsque la somme de deux bits de même poids dépasse 1 (chiffre maximum en binaire).

Voyons comment additionner en binaire les nombres 31 (011111) et 25 (011001)

	0	1	1	1	1	1
+	0	1	1	0	0	1
	1	1	1	0	0	0

Nombre négatif

Question : comment peut-on traiter les nombres négatif en informatique avec des « 0 » et des « 1 » ?

Réponse en utilisant le premier bit comme signe 0 → positif 1 → négatif

Problème : ce raisonnement s'applique dans certain cas d'opération mais pas dans toutes

Exemple :

$$\begin{array}{r} 1000\ 0111 = -7 \\ + \quad 0111\ 1110 = 126 \\ \hline 0000\ 0101 = 5 \rightarrow \text{résultat faux} \end{array}$$

On utilise le complément à 2 pour réaliser ces opérations

	Nombre		complément à 1		ajout de 1		complément à 2	
7	0000 0111	→	1111 1000	→	1111 1001	=	1111 1001	(-7)
126	0111 1110	----->				+	0111 1110	

(126)

(1) 0111 0111 = 119 OK

Multiplication binaire

La multiplication binaire s'effectue de la même façon qu'en notation décimale : un nombre multiplié par 0 est égal à 0. On a donc :

- $0*0=0$
- $0*1=0$
- $1*0=0$
- $1*1=1$

Essayons via cet exemple de multiplier 31 (11111) par 6 (110) :

			0	1	1	1	1	1	
	*					1	1	0	
			0	0	0	0	0	0	
		0	1	1	1	1	1		
0	+	1	1	1	1	1			
		1	0	1	1	1	0	1	0

ALGEBRE DE BOOLE

L'algèbre de Boole qui tire son nom du mathématicien anglais Georges Boole a pour fonction de traduire des signaux électriques en expressions mathématiques.

Ainsi, à chaque signal correspond une variable logique qui sera traitée par une fonction logique. Ces fonctions peuvent être simplifiées grâce à l'algèbre de Boole et à des identités remarquables de base.

On représente ainsi des circuits logiques grâce à des symboles avant de les réaliser physiquement.

Les expressions logiques sont utilisées en électronique, et donc dans nos PC. Un processeur par exemple est composé de transistors assemblés permettant de réaliser des fonctions très simples (addition, soustraction, etc.).

Variable logique :

Elle se note avec un caractère. Elle peut prendre deux valeurs logiques : 0 ou 1 (langage binaire, le seul compréhensible par l'ordinateur).

Fonction logique :

C'est une fonction permettant d'avoir plusieurs états de sortie en fonction des entrées.

Ex : $S=a+b+cd$

Portes logiques :

Les opérations simples faites par les transistors s'établissent à l'aide de portes logiques. Voici la liste des portes logiques :

- *La fonction **ET (AND)** : la sortie est égale à 1 si les deux variables sont égales à 1, sinon elle est égale à 0.
Elle se représente par un point (dans les fonctions logiques)*
- *La fonction **OU (OR)** : la sortie est égale à 0 si les deux variables sont à 0, sinon elle est égale à 1. Elle se représente par un signe + dans les fonctions logiques*
- *La fonction **NON (NO)** : elle est égale au contraire de la variable. Si la variable est à 0, la sortie sera à 1, et inversement.*
- *La fonction **OU EXCLUSIF (XOR)** : elle est égale à 1 si une des deux entrées sont égales à 1 mais pas les deux.*

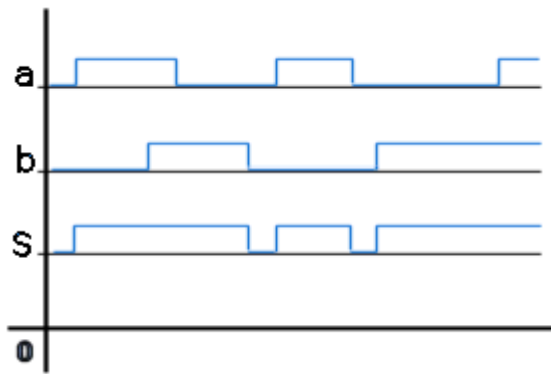
Les portes logiques sont aussi regroupées en deux portes principales utilisées en électronique :

- *Les portes **NAND (NON ET)** : elles sont le regroupement d'une fonction **NON** et d'une fonction **ET***
- *Les portes **NOR (NON OU)** : elles sont le regroupement d'une fonction **NON** et d'une fonction **OU***

Chronogramme :

L'état d'un signal se représente graphiquement à l'aide de chronogrammes, qui comme leur nom le stipule indiquent la valeur du signal en fonction du temps.

*Exemple d'un chronogramme avec la fonction **OU** :*



Fonctions logiques booléennes :

Les opérateurs des fonctions logiques sont représentés par des signes :

- L'opérateur **ET** se représente par un point
- L'opérateur **OU** se représente par un signe +
- L'opérateur **NON** se représente par un slash (/) devant la variable ou par une barre au-dessus de la variable : $\overline{A}$
- L'opérateur **OU exclusif (XOR)** se représente par un signe + encerclé

Table de vérité :

La table de vérité est un tableau rassemblant toutes les entrées avec toutes les possibilités de sorties (2^n variables pour le binaire)

Exemple de la table de vérité de la fonction **OU** :

Les lois de composition :

Les lois de composition sont des règles simples d'écriture permettant de simplifier les expressions algébriques booléennes.

- L'associativité : $A.(B.C)$ est équivalent à $B.(C.A)$ ou $C.(B.A)$
- L'absorption : $A.(A.B)$ est équivalent à $A.B$, de même que $A+/(A.B)$ est équivalent à $A+B$
- La commutativité : $A.B$ est équivalent à $B.A$. $A+B$ est équivalent à $B+A$
- La distributivité : $A.(B+C)$ est équivalent à $A.B + A.C$
- L'idempotence : $A.A$ est équivalent à A et $A+A$ est équivalent à A
- L'inversion : $A./A$ est équivalent à 0 . $A+/(A)$ est équivalent à 1
- L'identité : $A.1$ est équivalent à A . $A+0$ est équivalent à A . $A.0$ est équivalent à 0

Théorème de DE MORGAN :

$$A.B = \overline{\overline{A} + \overline{B}}$$

Exemple de simplification d'équation :

$$S = \overline{a} + \overline{b} + a.b.c + a.b.\overline{c} + \overline{a}.b.c$$

$$s = \overline{a}.(1+bc) + ab.(c+\overline{c}) + \overline{a}.b.c$$

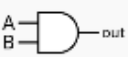
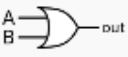
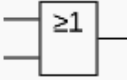
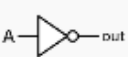
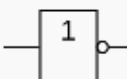
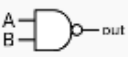
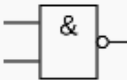
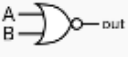
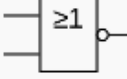
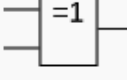
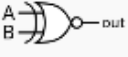
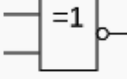
$$s = \overline{a} + ab + \overline{a}.b.c$$

$$s = \overline{a} + b + \overline{a}.b.c$$

$$s = 1$$

Portes logiques

Symboles et tables de vérité de portes logiques

Type	Symbole américain	Symbole française	Opération booléenne entre A et B	Table de vérité																		
ET			$A \cdot B$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A ET B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Entrée		Sortie	A	B	A ET B	0	0	0	0	1	0	1	0	0	1	1	1
Entrée		Sortie																				
A	B	A ET B																				
0	0	0																				
0	1	0																				
1	0	0																				
1	1	1																				
OU			$A + B$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A OU B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Entrée		Sortie	A	B	A OU B	0	0	0	0	1	1	1	0	1	1	1	1
Entrée		Sortie																				
A	B	A OU B																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	1																				
NON			$\overline{A}$	<table><tr><th>Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>NON A</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	Entrée	Sortie	A	NON A	0	1	1	0										
Entrée	Sortie																					
A	NON A																					
0	1																					
1	0																					
En électronique, une porte NON est plus communément appelée <i>Inverseur</i> . Le cercle utilisé sur la représentation est appelé « bulle », et on l'utilise généralement dans les diagrammes pour montrer qu'une entrée ou une sortie est inversée.																						
NON-ET (NAND)			$\overline{A \cdot B}$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A NAND B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Entrée		Sortie	A	B	A NAND B	0	0	1	0	1	1	1	0	1	1	1	0
Entrée		Sortie																				
A	B	A NAND B																				
0	0	1																				
0	1	1																				
1	0	1																				
1	1	0																				
NON-OU (NOR)			$\overline{A + B}$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A NOR B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Entrée		Sortie	A	B	A NOR B	0	0	1	0	1	0	1	0	0	1	1	0
Entrée		Sortie																				
A	B	A NOR B																				
0	0	1																				
0	1	0																				
1	0	0																				
1	1	0																				
OU exclusif (XOR)			$A \oplus B$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A XOR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	Entrée		Sortie	A	B	A XOR B	0	0	0	0	1	1	1	0	1	1	1	0
Entrée		Sortie																				
A	B	A XOR B																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	0																				
ET-Inclusif ou OU-exclusif complémenté (XNOR) ou Identité ou NON-OU exclusif			$A \odot B$	<table><tr><th colspan="2">Entrée</th><th>Sortie</th></tr><tr><th>A</th><th>B</th><th>A XNOR B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	Entrée		Sortie	A	B	A XNOR B	0	0	1	0	1	0	1	0	0	1	1	1
Entrée		Sortie																				
A	B	A XNOR B																				
0	0	1																				
0	1	0																				
1	0	0																				
1	1	1																				

Réalisation d'un ou-exclusif en utilisant des nand

a	b	S
0	0	0
0	1	1
1	0	1
1	1	0

$$S = \neg a.b + a.\neg b$$

$$S = \neg(\neg(a.b + a.\neg b))$$

$$S = \neg(\neg(\neg(a.b).\neg(a.\neg b)))$$

